

humand

Manual Web Api



Introducción

Humand ofrece un conjunto de integraciones que, a partir de ahora, denominaremos "WEB API". Estas integraciones están disponibles sin costo alguno para todos nuestros clientes y tienen como objetivo principal facilitar el acceso en tiempo real y de forma sincrónica a su información en nuestra plataforma.

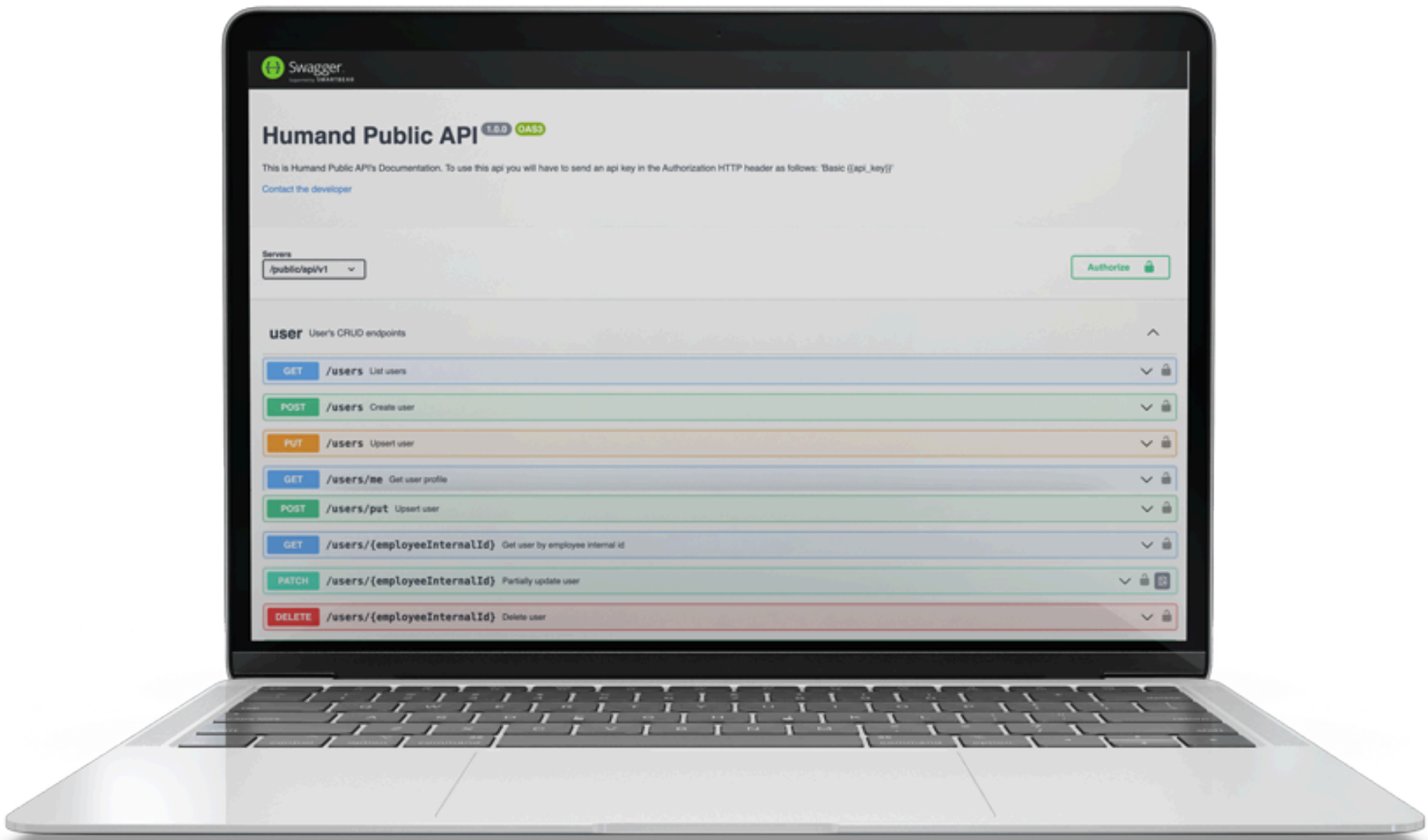
La finalidad principal de esta API es proporcionar una interfaz amigable y segura que pueda ser utilizada por cualquier programador. Además, está diseñada para simplificar la integración de nuestra solución en la nube con sus sistemas legados existentes, haciendo uso de una arquitectura ampliamente probada y difundida en la actualidad, como es el caso de API REST.

Para alcanzar esta meta, la WEB API proporciona una variedad de servicios estándar que incluyen la creación y obtención de empleados, la gestión de organigramas, la capacidad de realizar publicaciones, la carga de recibos de nómina, la segmentación de usuarios en grupos específicos, y mucho más.

Documentación detallada

Contamos con un sitio donde se puede consultar toda la información necesaria para crear una integración con nuestro sistema "WEB API" (ejemplos de invocación, respuesta y mucho más). Utilizamos una herramienta estándar en la industria para detallar toda esta información, como lo es Swagger.

La herramienta Swagger permite documentar y probar las APIs. Es así como no sólo se puede explorar la lista de servicios y su correspondiente información (cabecera de los mensajes, parámetros, estructura de las respuestas, etc.) sino que también se pueden probar los servicios a través del navegador mismo, sin necesidad de escribir código de programación.



Generalidades del procedimiento de Integración

A. El acceso a la plataforma de pruebas se otorga mediante el uso de una API Key

Esta información específica de acceso a la plataforma de pruebas se proporcionará por parte de Humand una vez que se tome la decisión de llevar a cabo la integración.

Una vez que se hayan completado satisfactoriamente las pruebas en el entorno de pruebas y la integración esté lista para su implementación en el entorno de producción, se realizará la conexión a la comunidad de producción utilizando una API Key diferente que será suministrada por nosotros.

Este proceso garantiza una transición segura y controlada desde el ambiente de pruebas al ambiente de producción, manteniendo la seguridad y la integridad de los datos en todo momento.

Actualmente, el acceso otorgado a través de una API Key es a la plataforma de testing (esta información se brindará desde Humand una vez que se decida por realizar la integración). Una vez finalizadas las pruebas, la conexión se realizará a la comunidad de producción con una API Key diferente que les proporcionaremos.

Importante

Esta API Key debe estar asociada a un usuario que cuenta con permisos de administrador. Es importante que este Usuario NO sea eliminado para asegurar la integración de forma permanente.

B. El ingreso de la API Key para realizar los tests es siguiendo este formato

Basic JyBpaEH7YMppAQjqyGMSV53PHk9-Pd_T

Siendo "JyBpaEH7YMppAQjqyGMSV53PHk9-Pd_T" la API Key asociada en este caso (el equipo de Humand proporcionará la correspondiente a cada comunidad). Notar que se debe introducir la palabra Basic seguido de un espacio y luego la API Key asociada.

Available authorizations

api_key (apiKey)

Basic {{api_key}}

Name: Authorization

In: header

Value:

Authorize

Close

Detalles del alcance de las integraciones que se pueden realizar a Humand

Los usuarios en Humand cuentan con tres tipos de información pertinente que componen su perfil

Campos básicos de información personal

Esta es la información básica que un usuario tiene en su perfil en la aplicación. Los datos obligatorios son con los que debe contar sí o sí el usuario para ser creado exitosamente en la plataforma. Los campos optativos, pueden o no ser agregados dependiendo de la información con la que contemos en la base de datos.



Nombre de usuario

Obligatorio, string - **"employeeInternalId": "string"**

Aclaración: NO es el nombre de la persona, es el usuario por el cual entrará a Humand.



Nombre de la persona

Obligatorio, string - **"firstName": "User first name"**

No se distingue entre primer y segundo nombre, iría completo.



Apellido

Obligatorio, string - **"lastName": "User last name"**

No se distingue entre primer y segundo apellido, iría completo..



Contraseña inicial

Obligatorio, string - **"password": "password"**

Debe tener como mínimo 8 caracteres.



Jefe directo

Optativo, string

En el json debe completarse el "employeeInternalId" del usuario asignado como jefe. El jefe directo se completa con la siguiente parte del json:

```
"relationships": [  
  {  
    "name": "BOSS",  
    "employeeInternalId": "userEmployeeInternalId"  
  }  
]
```

Importante

Siempre hay que generar primero el usuario del jefe con su employeeInternalId correspondiente y luego, al momento de generar el usuario del colaborador subordinado, colocar como employeeInternalId el useremployeeInternalId de su jefe. Ya que si se coloca como jefe un employeeinternalId que todavía no existe en la comunidad, surgirá un error.

- > **Mail del usuario**
Optativo, string - **"email": "user@email.com",**
- > **Apodo del usuario**
Optativo, string - **"nickname": "userNickName",**
- > **Teléfono del usuario**
Optativo, string - **"phoneNumber": "+5491123456789",**
Es necesario poner el prefijo del país
- > **Fecha de contratación del usuario**
Optativo, string - **"hiringDate": "2022-01-01",**
- > **Fecha de nacimiento del usuario**
Optativo, string - **"birthdate": "2022-01-01"**

Segmentación asociada al usuario

Estas propiedades nos permiten agrupar a los usuarios de acuerdo a características en común que tengan, como por ejemplo pertenecer a una misma área o trabajar en una misma ubicación. Las segmentaciones están formadas por:

- > **Grupo de segmentación**
"group": "Segmentation group name"
Cada grupo tiene un único título y un usuario puede pertenecer a varios grupos de segmentación.
- > **Ítems de segmentación**
"item": "Segmentation item name"
Cada grupo tiene uno o más ítems de segmentación. Cada usuario puede tener ningún, uno o múltiples ítems para un grupo de segmentación en particular.

Las segmentaciones no son de carácter obligatorio, dependen de la información con la cual contamos en la base de datos, algunos ejemplos pueden ser: Gerencia, Sector, Ubicación

La sección del json correspondiente a la segmentación de un usuario, al momento de su creación, sería:

```
"segmentation": [  
  {  
    "group": "Gerencia",  
    "item": "Segmentation item name"  
  },  
  {  
    "group": "Sector",  
    "item": "Segmentation item name"  
  },  
  {  
    "group": "Ubicación",  
    "item": "Segmentation item name"  
  },  
]
```

Campos dinámicos o extras del perfil de los usuarios

Esta es la información adicional que cada comunidad puede elegir crear para que los usuarios tengan. Los datos son optativos y pueden o no ser agregados dependiendo de la información con la que contemos en la base de datos.

Importante
Los campos de perfil que se escriban en la API deberán coincidir al 100% (respetando las mayúsculas y minúsculas) del campo ingresado en la parte de Ajustes -> Perfiles dentro del Panel Administrador de Humand.

> **Identificador único del Valor del campo dinámico (se podrá adquirir mediante GET)**
"ID": "7b122126-f8ec-4410-b163-8389ed21e79f"

> **Nombre del campo de perfil**
"name": "Dynamic field name"

Es importante tener en cuenta que se puede enviar ID o Name, no es necesario mandar ambos campos. En el caso de enviar ambos campos, es necesario enviarlo con el valor del mismo (no vacío).

> **Información a completar del campo de perfil**
"value": "Dynamic field value"

Dentro del json siguen la siguiente estructura general:

```
"fields": [  
  {  
    "ID": "7b122126-f8ec-4410-b163-8389ed21e79f",  
    "name": "Dynamic field name",  
    "value": "Dynamic field value"  
  }  
]
```

Campos dinámicos de ejemplo pueden ser: Puesto, Número de Interno, Gerencia, Edificio, Legajo. Considerando esto, la sección completa de esta parte del json de creación de un usuario asociada a los campos dinámicos, sería:

```
"fields": [  
  {  
    "name": "Puesto",  
    "value": "Dynamic field value"  
  },  
  {  
    "name": "Número de Interno",  
    "value": "Dynamic field value"  
  },  
  {  
    "name": "Gerencia",  
    "value": "Dynamic field value"  
  },  
  {  
    "name": "Edificio",  
    "value": "Dynamic field value"  
  },  
  {  
    "name": "Fecha de Inicio Puesto Actual",  
    "value": "Dynamic field value"  
  }  
],
```


Diferencia principal entre la Segmentación asociada al usuario y los Campos dinámicos o extras del perfil de los usuarios:

La gran diferencia entre estos dos tipos de información es que la Segmentación asociada al usuario actúa como un filtro en Humand. Un ejemplo sería que podríamos filtrar un posteo según la ubicación de los colaboradores de la empresa, siendo la Ubicación una de esas Segmentaciones. En cambio, los Campos dinámicos de perfil son detalles que aparecen en los perfiles de los colaboradores, un ejemplo de esto sería su cédula de identidad. Es decir que es un campo informativo que no se utiliza para filtrar.

A tener en cuenta

Antes de enviar las segmentaciones o los campos dinámicos de perfil a través de la API, te recomendamos ingresar a admin.humand.co con las credenciales de la comunidad de Testing asignadas.

[Hu Admin.](#)

Encontrarás la sección llamada "Segmentación". En este apartado, podrás cargar previamente las distintas segmentaciones y los ítems asociados a cada una. Estos son personalizables y se pueden ajustar según la base de datos que estés utilizando. Puedes realizar esta carga de manera masiva.

[Tutorial](#)

Aclaración: verás que en los videos se habla sobre la sección de "Grupos" dentro del panel administrador. Actualmente, el nombre de esa sección fue modificada por "Segmentación".

Otra sección que encontrarás será la de Ajustes. Una vez allí, te pedimos que te dirijas a la tercera pestaña llamada Perfiles. Al pie de la página, encontrarás una sección para cargar los campos dinámicos de perfil donde podrás agregar tantos como sea necesario. Los mismos son customizables y se pueden ajustar según la base de datos que estés utilizando. Aquí encontrarás un tutorial que explica cómo hacerlo, a partir del minuto 2.02.

[Tutorial](#)

Es esencial que la nomenclatura de las segmentaciones y campos dinámicos dentro del panel administrador de Humand, coincida con la que se envía a través de la API.

Usos de la API

Usuarios

POST - Create Users: este endpoint lo utilizaremos para la creación de nuevos usuarios. A continuación compartimos el ejemplo del json para el caso completo de un usuario con los ejemplos brindados (todas las segmentaciones y todos los campos dinámicos incluídos)

Aclaración: Cuando se sube un mail mediante POST se envía un mail automático a ese correo invitando a unirse a la comunidad digital. Si no quieren que esto suceda es necesario subirlo mediante PUT o PATCH.

```

{
  "password": "password",
  "segmentation": [
    {
      "group": "Gerencia",
      "item": "Segmentation item name"
    },
    {
      "group": "Sector",
      "item": "Segmentation item name"
    },
    {
      "group": "Ubicación",
      "item": "Segmentation item name"
    },
  ],

  "relationships": [
    {
      "name": "BOSS",
      "employeeInternalId": "userEmployeeInternalId"
    }
  ],

  "fields": [
    {
      "name": "Puesto",
      "value": "Dynamic field value"
    },
    {
      "name": "Número de Interno",
      "value": "Dynamic field value"
    },
    {
      "name": "Gerencia",
      "value": "Dynamic field value"
    },
    {
      "name": "Edificio",
      "value": "Dynamic field value"
    },
    {
      "name": "Fecha de Inicio Puesto Actual",
      "value": "Dynamic field value"
    }
  ],
  "employeeInternalId": "string",
  "email": "user@email.com",
  "firstName": "User first name",
  "lastName": "User last name",
  "nickname": "userNickName",
  "phoneNumber": "5491123456789",
  "hiringDate": "2022-01-01",
  "birthdate": "2022-01-01"
}

```

PUT - Upsert Users: este endpoint lo utilizaremos para la actualización de usuarios existentes. El json que debe enviarse es el completo de la creación del usuario, incluyendo los campos modificados. El body del json en este caso es igual que en el del POST. Importante: Para asegurarse de que no se borre información, es fundamental que el body del PUT contenga toda la información del usuario (incluso si esta no fue modificada)

PATCH - Partially update user: este endpoint lo utilizaremos para la actualización parcial de usuarios existentes, desde el mismo no se podrán modificar los campos dinámicos de perfil. El json debe enviarse con la información que se quiere modificar solamente.


```

{
  "segmentation": [
    {
      "group": "Gerencia",
      "item": "Segmentation item name"
    },
    {
      "group": "Sector",
      "item": "Segmentation item name"
    },
    {
      "group": "Ubicación",
      "item": "Segmentation item name"
    },
  ],
],
"employeeInternalId": "string",
"email": "user@email.com",
"firstName": "User first name",
"lastName": "User last name",
"nickname": "userNickName",
"phoneNumber": "5491123456789",
"hiringDate": "2022-01-01",
"birthdate": "2022-01-01"
}

```

PATCH - Update user profile fields: este endpoint lo utilizaremos para la actualización parcial de campos dinámicos de perfil de usuarios existentes. Es importante que el json solo debe enviarse con la información que se quiere modificar.

```

"fields": [
  {
    "name": "Puesto",
    "value": "Dynamic field value"
  },
  {
    "name": "Número de Interno",
    "value": "Dynamic field value"
  },
  {
    "name": "Gerencia",
    "value": "Dynamic field value"
  },
  {
    "name": "Edificio",
    "value": "Dynamic field value"
  },
  {
    "name": "Fecha de Inicio Puesto Actual",
    "value": "Dynamic field value"
  }
],

```

PATCH & POST - Partially update user segmentation: Estos Endpoints sirven para actualizar parcialmente la segmentación de un usuario. En el PATCH anterior correspondiente a la actualización del usuario, el bloque de segmentación se debe mandar completo para actualizar la información del usuario, es decir, se deben enviar todas las segmentaciones e ítems a las que pertenece el mismo. En el caso de querer actualizar parcialmente la segmentación de un usuario, se cuenta con 2 nuevos métodos de un nuevo endpoint.

POST - /users/:employeeInternalId/segmentations: Este endpoint lo utilizaremos para asignar al usuario los ítems del segmento que se envían, sin necesidad de mandar el detalle completo de los otros ítems o segmentos.

Importante: esto aplica para asignaciones nuevas, es decir, si ese usuario formaba parte de otro ítem del mismo segmento no lo "reemplaza" (en ese caso ver método siguiente)

Ejemplo: Supongamos que tenemos dos segmentos: 'Ubicación' y 'Tareas'. El segmento 'Ubicación' contiene los ítems 'Localidad A', 'Localidad B', y 'Localidad C'. El segmento 'Tareas' incluye los ítems 'Administrativo', 'Comercial' y 'Operativo'.

Ahora, consideremos un usuario X que forma parte del ítem 'Localidad A' en el segmento 'Ubicación' y del ítem 'Administrativo' en el segmento 'Tareas'. Si enviamos una solicitud a este endpoint para agregar el ítem 'Comercial' del segmento 'Tareas' al usuario X, este se añadirá a dicho ítem. El usuario X permanecerá también en 'Administrativo' del segmento 'Tareas' y en 'Localidad A' del segmento 'Ubicación'.

```
{
  "segmentations": [
    {
      "group": "Segmentation group name",
      "item": "Segmentation item name"
    }
  ]
}
```

PATCH - /users/:employeeInternalId/segmentations: Este endpoint lo utilizaremos para reemplazar los ítems que se envían al usuario de una misma segmentación, sin modificar las segmentaciones que no se envían.

Ejemplo: Supongamos que tenemos dos segmentos: 'Ubicación' y 'Tareas'. El segmento 'Ubicación' contiene los ítems 'Localidad A', 'Localidad B' y 'Localidad C'. El segmento 'Tareas' incluye los ítems 'Administrativo', 'Comercial' y 'Operativo'.

Consideremos un usuario X que forma parte del ítem 'Localidad A' en el segmento 'Ubicación' y del ítem 'Administrativo' en el segmento 'Tareas'. Si enviamos una solicitud a este endpoint para agregar el ítem 'Comercial' del segmento 'Tareas' al usuario X, este se añadirá a dicho ítem y ya no formará parte del ítem 'Administrativo' del mismo segmento, aunque seguirá perteneciendo a 'Localidad A' del segmento 'Ubicación'.

```
{
  "segmentations": [
    {
      "group": "Segmentation group name",
      "item": "Segmentation item name"
    }
  ]
}
```

DELETE - Delete Users: en este caso, el endpoint será utilizado para eliminar usuarios. Para ello lo único que debemos compartir es el nombre del usuario (employeeInternalId).

Segmentaciones

GET - Segmentations: Este endpoint permite obtener el detalle de todas las segmentaciones disponibles incluyendo información sobre cada ítem asociado.

POST - Segmentations: Este caso será utilizado en caso de que se desee agregar una segmentación nueva. Para ello el esquema a seguir es:

```
{
  "name": "Segmentation group name",
  "itemNames": [
    "Segmentation item name"
  ],
  "visibility": "ALL"
}
```

Segmentation group name

Tipo: string

Descripción: El nombre de la propiedad que permite agrupar usuarios de acuerdo a características en comun. El mismo tiene que ser único y cada usuario puede pertenecer a varios grupos.

Segmentation item name

Tipo: string

Descripción: Campos asociados a esos grupos para dividir usuarios según características específicas.

Visibility

Tipo: string

Descripción: Este campo nos sirve para determinar el nivel de visibilidad de un grupo de segmentación. Puede ser ALL (para todos los usuarios), ADMINS_ONLY (para los administradores nada más), USER_AND_ADMINS (para el propio usuario y administradores).

PUT - Segmentations: Si quisiéramos modificar las segmentaciones o ítems actuales, deberíamos utilizar el endpoint de PUT Segmentations con el json completo correspondiente (similar al anterior).

POST - Segmentations/groups/{groupName}/items: Este endpoint permite agregar ítems a una segmentación existente. Para hacerlo, este es el cuerpo a enviar en la solicitud

Cuerpo de la solicitud (Request body):

```
{
  "items": [
    "Segmentation item name"
  ]
}
```

Para agregar múltiples ítems, simplemente incluye cada uno como una entrada separada por comas dentro de la lista "items".

```
{
  "items": [
    "Segmentation item name 1",
    "Segmentation item name 2",
    "Segmentation item name 3"
  ]
}
```

Ejemplo - Si el grupo de Segmentación es Ubicación los items que podrían usarse son:

- Guadalajara
- Buenos Aires
- Montevideo
- Santiago de Chile

Documentos Personales

POST - Document: Desde este endpoint se podrán subir documentos personales (recibos de nómina, contratos, entre otros) a los usuarios.

Algunos de los campos que podrán completar serán los siguientes:

file

Tipo: string (binary)

Descripción: En este campo se debe enviar el documento en formato binario, lo que significa que el archivo se adjunta en su formato crudo, sin procesamiento.

folderId

Tipo: integer

Descripción: El 'folderId' es un identificador numérico que corresponde a la carpeta donde se almacenará la información. Para encontrar este valor, se debe acceder a la plataforma desde la cuenta de administrador y navegar a la sección 'Documentos personales'. El 'folderId' se encuentra en la URL de la carpeta en la que se desea almacenar la información. Este identificador se mantiene constante a lo largo del tiempo, es decir que siempre que un archivo específico vaya a una carpeta en particular el ID de esa carpeta se mantiene. Los colaboradores no podrán ver los archivos de otros usuarios.

En el siguiente ejemplo el folder id sería: 14326



name

Tipo: string

Descripción: Este campo se utiliza para especificar el nombre del archivo que se va a enviar, el mismo debe tener una nomenclatura única. Se recomienda que el nombre del archivo siga una nomenclatura que incluya el 'internalId' del colaborador.

sendNotification

Tipo: boolean

Descripción: Este campo se utiliza para indicar si se debe enviar una notificación cuando se envía el archivo. Si el valor es 'true', se enviará una notificación. Si es 'false', no se enviará notificación.

signatureStatus

Tipo: string

Descripción: Este campo se utiliza para indicar si se requiere una firma en el archivo. Se debe especificar el estado de la firma, por ejemplo, si es necesaria o no. Las posibles opciones son: SIGNED_IN_DISAGREEMENT, SIGNATURE_NOT_NEEDED, PENDING, SIGNED, SIGNED_IN_AGREEMENT

signatureCoordinates

Tipo: objeto

Descripción: En este campo, se solicita que expongamos las coordenadas en donde deberían estar las firmas. Si es necesaria la firma pueden enviarle un template del recibo a su referente de Humand y pronto le comentaremos las coordenadas.

allowDisagreement

Tipo: boolean

Descripción: Este campo se utiliza para determinar si se permite firmar con disconformidad.

Un ejemplo de un Json a enviar podría ser el siguiente:


```
{
'folderId': '1',
'name': 'File name',
'sendNotification': 'false',
'signatureStatus': 'PENDING',
'signatureCoordinates':
'[{ "page":0,"x":0.09078528515064561,"y":0.7916669970581939,"height":0.05259172329577442,"width":
0.21792615674318508}]',
'allowDisagreement': 'false'
}
```

En el proceso de envío de archivos, es fundamental entender que no se deben enviar los datos en formato JSON, sino en un formato conocido como 'form data'. Para lograr esto, debemos realizar los siguientes pasos además de lo anteriormente mencionado:

Asegurarse de que en los encabezados (headers) de la solicitud se incluya la clave 'Content-Type' con el valor 'multipart/form-data'. Esto indica que estamos enviando datos en formato 'form data'.

Además de los campos y valores que se describen en el JSON, es necesario agregar el archivo que deseamos enviar. Esto se hace utilizando la clave 'file' para identificar el archivo adjunto.

Siguiendo estos pasos, estaremos listos para enviar archivos de manera adecuada a través de solicitudes HTTP utilizando 'form data', garantizando así una comunicación efectiva con el servidor.

Un ejemplo en Python de todo completo:

```
import requests
```

```
url = "https://api-prod.humand.co/public/api/v1/users/Employee_internal_id/documents/files"
```

```
payload = {'folderId': '1',
'name': 'File name',
'sendNotification': 'false',
'signatureStatus': 'PENDING',
'signatureCoordinates':
'[{ "page":0,"x":0.09078528515064561,"y":0.7916669970581939,"height":0.05259172329577442,"width":
0.21792615674318508}]',
'allowDisagreement': 'false'}
files=[
('file',('document.pdf',open('/Users/username/Downloads/document.pdf','rb'),'application/pdf'))
]
headers = {
'Content-Type': 'multipart/form-data',
'Accept': 'application/json',
'Authorization': 'API_KEY'
}
```

```
response = requests.request("POST", url, headers=headers, data=payload, files=files)
```

```
print(response.text)
```

Un ejemplo en Java de todo completo:

```
var myHeaders = new Headers();
myHeaders.append("Content-Type", "multipart/form-data");
myHeaders.append("Accept", "application/json");
myHeaders.append("Authorization", API_KEY);

var formdata = new FormData();
formdata.append("file", "voluptate adipisicing Duis veniam enim");
formdata.append("folderId", "1");
formdata.append("name", "File name");
formdata.append("sendNotification", "false");
formdata.append("signatureStatus", "PENDING");
formdata.append("signatureCoordinates", "[object Object]");
```


Vacaciones

GET - Get time off requests: Desde este endpoint se podrán obtener las solicitudes de Vacaciones y permisos que se han realizado en Humand.

Alguno de los campos a completar son los siguientes:

Page

Tipo: integer
Descripción: El campo 'page' especifica el número de la página actual que se está solicitando. Es obligatorio completar esta información. Por ejemplo "page=2" indica que se está solicitando la segunda página de resultados.

A continuación dejamos una imagen a modo de ilustración:

registro 1	page 1
registro 2	limit 10
registro 3	
registro 4	
registro 5	
registro 6	
registro 7	
registro 8	
registro 9	
registro 10	
registro 11	page 2
registro 12	limit 10
registro 13	
registro 14	
registro 15	
registro 16	
registro 17	
registro 18	
registro 19	
registro 20	
registro 21	page 3
registro 22	limit 10
registro 23	
registro 24	
registro 25	
registro 26	
registro 27	
registro 28	
registro 29	
registro 30	

Limit

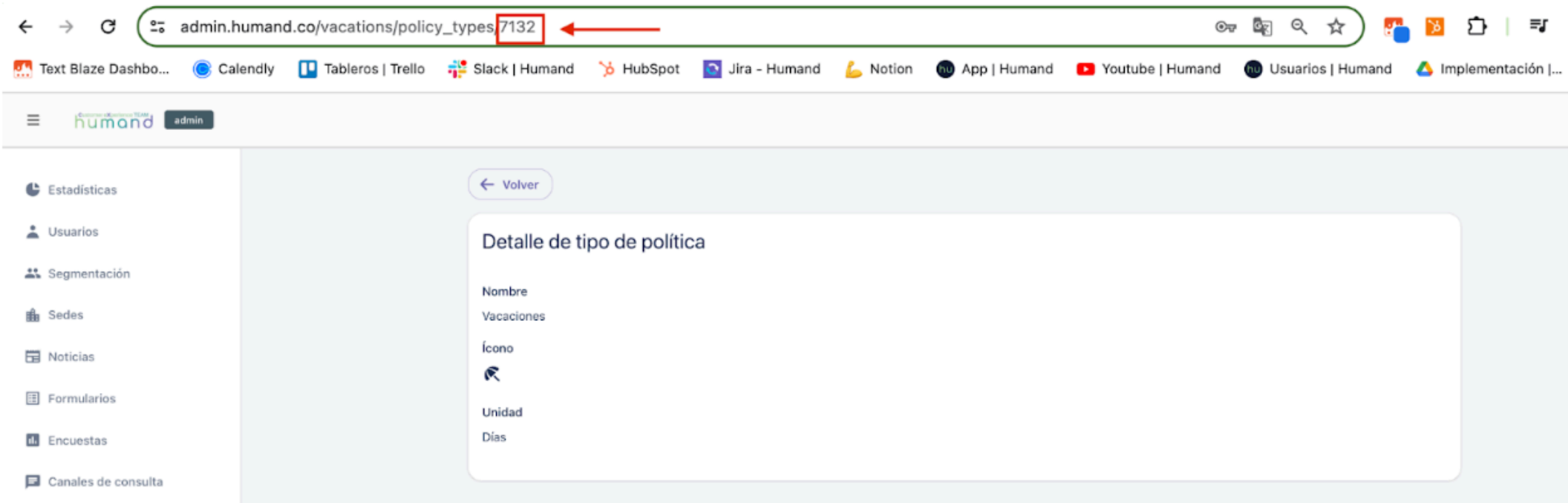
Tipo: integer
Descripción: Este campo define la cantidad máxima de elementos o registros que se desean obtener por página, el valor máximo permitido es 500, controlando así el tamaño de cada segmento de datos entregado. Es obligatorio completar esta información. Por ejemplo "limit=10" indica que se desean obtener hasta 10 registros por página.

States

Tipo: string
Descripción: Este campo da la posibilidad de especificar el estado de la solicitud a la hora de obtener la información. Las posibilidades son las que fueron aprobadas, las que están en progreso, las rechazadas y las canceladas. Es opcional. APPROVED, REJECTED, IN_PROGRESS, CANCELLED.

PolicyTypelds

Tipo: string
Descripción: En este campo se podrá elegir sobre qué tipo política es la información sobre las solicitudes realizadas, Vacaciones, Día de estudio, Licencia por maternidad.
Para poder encontrar el ID de la política lo podrás hacer viendo los últimos números que figuran en la URL cuando se ingresa en el detalle de la misma:



from.date

Tipo: string
Descripción: En este campo podrás obtener las solicitudes cuyos días están contenidas a partir de la fecha seleccionada.

to.date

Tipo: string
Descripción: En este campo podrás obtener las solicitudes cuyos días están contenidas hasta la fecha seleccionada.

resolutionDateFrom

Tipo: string
Descripción: Fecha "desde" para la fecha que se aprobó, rechazó o canceló esa solicitud

resolutionDateTo

Tipo: string
Descripción: Fecha "hasta" para la fecha que se aprobó, rechazó o canceló ese pedido esa solicitud

Un ejemplo de un Json podría ser el siguiente:

```
{
  "page": 1,
  "limit": 100,
  "states": "APPROVED",
  "PolicyTypeIds": "1,2,3",
  "fromDate": "2023-01-01",
  "toDate": "2023-12-31"
}
```

POST - Manual correction of time off balances: Desde este endpoint podrás ajustar los saldos de las vacaciones de los colaboradores sumando o restando días disponibles.

userEmployeeInternalId

Tipo: string
Descripción: Es el nombre de usuario al que se le editarán días en las políticas de Vacaciones. Es obligatorio completar esta información.

operation

Tipo: string
Descripción: Este campo sirve para decidir si se suman o se restan días. Las posibilidades de elección son "ADDITION" o "SUBTRACTION". Es obligatorio completar esta información.

amount

Tipo: double
Descripción: Este campo es el resultado de días a agregar o quitar en una política de Vacaciones. Es obligatorio completar esta información.

observations

Tipo: string
Descripción: Es un campo visible solo en la base de datos. Sirve para subir la descripción de la adición o substracción de saldos. Es opcional completar esta información.

Un posible Json sería el siguiente:

```
{
  "employeeInternalId": "userEmployeeInternalId",
  "operation": "ADDITION",
  "amount": 1,
  "observations": "Observations"
}
```

Formularios

GET - Forms: Desde este endpoint se podrán obtener los resultados de un formularios en particular.

ID
Tipo: Integer
Descripción: El 'Id' es un identificador numérico que corresponde a cada uno de los formularios. Para encontrar este valor, se debe acceder a la plataforma desde la cuenta de administrador y navegar a la sección Formularios. El 'ID' se encuentra en la URL del Formulario, al mismo se accede haciendo click en Editar:

FormulariosRespuestas

+ Nuevo formulario

Buscar por título

Título	Estado	Creado en	Creador	Posición
Postulación a nuevo cargo	Activo	23/02/2022	Comunicaciones Empresar...	13
Referencia de un colega	Activo	23/02/2022	Comunicaciones Empresar...	
Afiliación de nuevo empleado	Activo	23/02/2022	Comunicaciones Empresar...	
Abre tu tienda online	Activo	23/02/2022	Comunicaciones Empresar...	

Editar

Duplicar

Eliminar

Descargar Excel

Descargar Excel filtrado

Es importante tener en cuenta que este identificador se mantiene constante a lo largo del tiempo.

admin.humand.co/forms/1465536/edit

shbo...CalendlyTableros | TrelloSlack | HumandHubSpotJira

admin

←

Postulación a nuevo cargo

CONTENIDOAJUSTESAPROBACI

En el siguiente ejemplo el forms ID sería: 1465536

page

Tipo: Integer

Descripción: El campo 'page' especifica el número de la página actual que se está solicitando. Es obligatorio completar esta información. Por ejemplo "page=4" indica que se está solicitando la cuarta página de resultados.

limit

Tipo: Integer
Descripción: Este campo define la cantidad máxima de elementos o registros que se desean obtener por página, el valor máximo permitido es 500, controlando así el tamaño de cada segmento de datos entregado. Es obligatorio completar esta información. Por ejemplo "limit=10" indica que se desean obtener hasta 10 registros por página.

order:

Tipo: String

Descripción: Este campo sirve para ordenar las respuestas de los formularios y así ver cuales fueron Rechazados y cuales fueron aprobados.
Las posibles respuestas son las siguientes: ASC, DESC, ASC_NULLS_FIRST, DESC_NULLS_LAST

orderBy

Tipo: String

Descripción: Este campo sirve para ordenar las respuestas según quien respondió el formulario o bien según su fecha de completado.
Las posibles respuestas son las siguientes: CREATOR, CREATED_AT

creationDate

Tipo: String

Descripción: Este campo sirve para buscar respuestas en una fecha determinada. La manera de escribir la fecha es YYYY-MM-DD

Un posible Json sería el siguiente:

```
{
  "id": 1465536,
  "page": 1,
  "limit": 100,
  "order": "ASC",
  "orderBy": "CREATED_AT",
  "creationDate": "2024-05-02",
}
```



El Control de Asistencia en Humand (Time Tracking) cuenta con dos APIs diseñadas para recibir registros horarios, integrándolos al sistema de control horario. Esto permite que dichos registros puedan ser utilizados dentro de la plataforma y sus reportes.

Endpoints disponibles

- POST /time-tracking/entries/clockIn: Recibe registros de inicio de jornada (Clock-in).
- POST /time-tracking/entries/clockOut: Recibe registros de fin de jornada (Clock-out).

Funcionalidades generales

- Sincronización individual: Cada request debe incluir un único registro horario en el body, ya que no es un método bulk.
- Soporte para registros pasados y futuros: Es posible sincronizar registros pasados o futuros, respetando siempre el orden: primero el registro de tipo START y luego el registro de tipo END.
- Timezones:
 - Los registros enviados deben incluir el timezone correspondiente en formato ISO 8601.
 - Ejemplo para Argentina (UTC -3): "2024-09-23T15:00:00.010-03:00".
 - El sistema reflejará la información en función del timezone configurado en la comunidad.
- Validaciones:
 - No se permite crear registros con el mismo tipo (Clock-in o Clock-out) consecutivamente en el mismo día para un empleado.
 - Los registros de entrada y salida deben estar emparejados (entrada + salida).

POST - /time-tracking/entries/clockIn - Clock-in entries

Recibe registros de tipo **START** (inicio de jornada laboral) para cualquier colaborador.

employeeId

Tipo: Integer

Descripción: Es el campo "Usuario" configurado dentro de la comunidad

Now

Tipo: Timestamp

Descripción: Timestamp del registro horario en formato ISO 8601, incluyendo fecha, hora y huso horario.

Comment

Tipo: String

Descripción: Es un campo libre utilizado para adicionar algún tipo de identificador de la terminal utilizada que luego será incluida como un comentario del registro horario dentro del sistema.

Un posible Json sería el siguiente:

```
{
  "employeeId": 1,
  "now": "2021-07-10T22:50:00.010Z",
  "comment": "string"
}
```

POST -/time-tracking/entries/clockOut - Clock-out entries

Recibe registros de tipo **END** (fin de jornada laboral) para cualquier colaborador.

employeeId

Tipo: Integer

Descripción: Es el campo "Usuario" configurado dentro de la comunidad

Now

Tipo: Timestamp

Descripción: Es el registro horario a sincronizar UTC-0

Comment

Tipo: String

Descripción: Es un campo libre utilizado para adicionar algún tipo de identificador de la terminal utilizada que luego será incluida como un comentario del registro horario dentro del sistema.

Un posible Json sería el siguiente:

```
{
  "employeeId": 1,
  "now": "2021-07-10T22:50:00.010Z",
  "comment": "string"
}
```

Cuestiones generales a considerar

Esta integración no es un plug&play con terminales biométricas. Esto quiere decir que se requiere de alguna interfaz intermediaria que fromatee los registros horarios y posteriormente mediante algún script o .net los impacte recurrentemente en los endpoints disponibilizados.

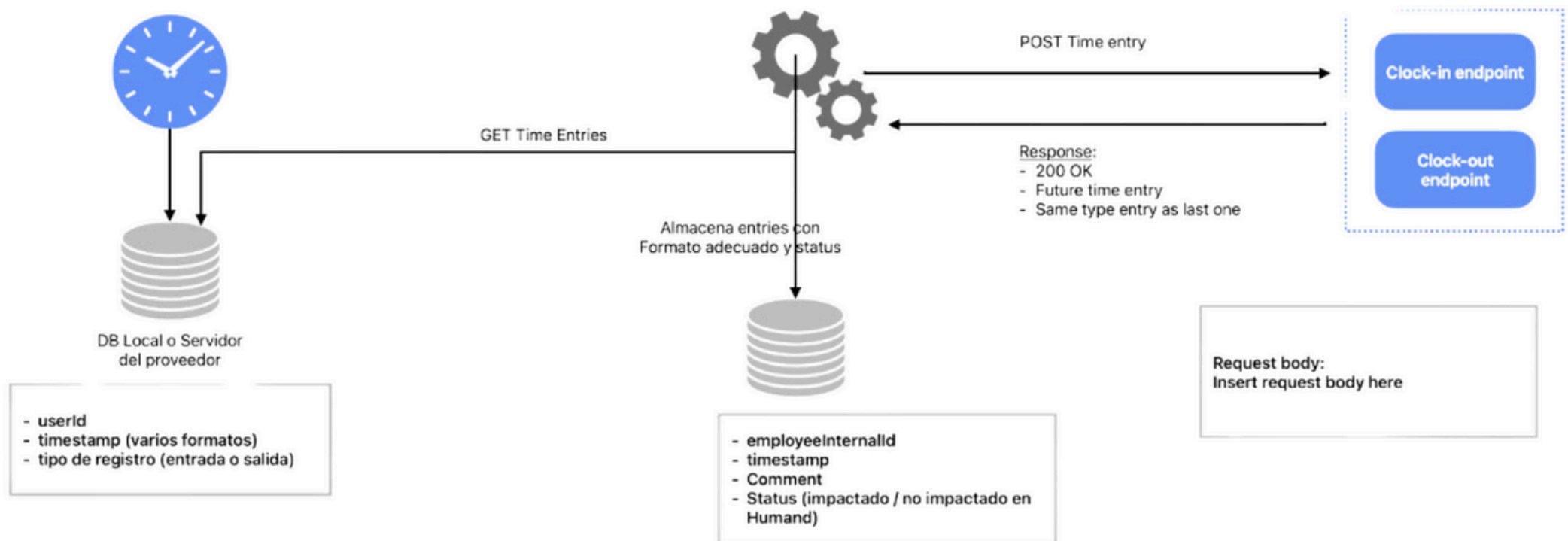
Diagrama - Ejemplo

A continuación se muestra un diagrama a modo de ejemplo de cómo podría verse el flujo desde una terminal biométrica propia hacia los endpoints de Humand. Notar que no es la única forma de realizar esta conexión si no que solamente una de muchas alternativas posibles en función de los requerimientos y recursos con los que cuenten.

RELOJ BIOMÉTRICO

INTERFAZ INTEGRACIÓN

HUMAND



Planificación horaria

Humand cuenta con dos métodos complementarios de planificación horaria:

- **Horarios laborales (Admin) = Work Schedules**
- **Turnos (Web) = Shift Management**

Ambos métodos de planificación horaria son compatibles entre sí dentro de las configuraciones de una comunidad y pueden utilizarse de manera complementaria. De esta forma, podemos optar si utilizar horarios laborales para algunos colaboradores y turnos para otros según el tipo de planificación horaria que tenga.

¿Cuándo usar un sistema u otro?

- **Horarios Laborales** → Utilizaremos horarios laborales cuando las configuraciones horarias sean más estáticas y se mantengan en un período de tiempo. El módulo igualmente permite hacer cambio entre horarios según la fecha de asignación, pero está pensado para otros más fijos como administrativos u operativos pero sin rotación. **Estos horarios los configura un administrador, no un jefe.**
- **Turnos** → Utilizaremos turnos cuando las configuraciones horarias sean fluctuantes o rotativas lo cual hace que sea muy complejo configurar esto mismo dentro de horarios laborales. El módulo está orientado para que los jefes puedan crear la planilla horaria de sus colaboradores y publicarla semana a semana una vez esté definida. **El módulo permitirá configurar rotaciones automáticas, modificaciones específicas por día, copiado y pegado de planificaciones a otros períodos, etc.**

Turnos laborales - Integración

Shifts es una herramienta complementaria a la arquitectura actual de horarios (Work Schedules), diseñada para ofrecer mayor flexibilidad en la planificación horaria de colaboradores. Introduce nuevas capacidades como la asignación puntual de turnos, planificación recurrente y autonomía para líderes de equipo, además de permitir integraciones externas más complejas.

En el caso de requerir integrar los turnos laborales de los colaboradores, utilizaremos este módulo que nos permite recibir de forma flexible los turnos de cada colaborador por fecha y por ID de colaborador.

Para utilizar este recurso, el usuario debe contar con el permiso de administrador de "Gestionar turnos" dentro de Humand Admin.

Endpoint POST Shifts - /shifts/bulk-create

Mediante este endpoint es posible que los clientes le indiquen al sistema los horarios de cada colaborador por cada fecha calendario. De esta forma, es posible configurar planificaciones horarias complejas que anteriormente no podían realizarse desde horarios laborales.

Algunos ejemplos de lo que Shift Management habilita: configurar rotaciones horarias, realizar modificaciones para un único colaborador en un día específico, sobrescribir un horario laboral con un cambio específico para un día y permitir que los jefes configuren el horario de su equipo únicamente, entre otros.

Un ejemplo de json (body) que recibe el request sería el siguiente:

```
{
  "employees": [
    {
      "employeeId": "string",
      "days": [
        {
          "date": "2025-05-23",
          "timeSlots": [
            "09:00",
            "10:00"
          ]
        },
        {
          "date": "2025-05-24",
          "timeSlots": [
            "11:00",
            "12:00"
          ]
        }
      ],
      "isWorkday": true
    }
  ]
}
```

employees

Tipo: Array

Descripción: Lista de colaboradores para los que se configurarán los turnos de trabajo.

employeeId

Tipo: String

Descripción: Es el campo "Usuario" configurado dentro de la comunidad.

days

Tipo: Array

Descripción: Lista de días con sus respectivos horarios de trabajo.

date

Tipo: Date (YYYY-MM-DD)

Descripción: Fecha en la que se computarán los turnos laborales.

timeSlots

Tipo: Array de arrays de String

Descripción: Intervalos de horario para ese día. Se puede configurar uno o varios timeSlots. Los horarios del mismo turno se deben separar utilizando coma (,), y los turnos entre sí se separan utilizando punto y coma (;):

Ejemplo: 09:00, 12:00 ; 13:00, 18:00

isWorkday

Tipo: Boolean

Descripción: Indica si el día debe ser considerado como día laborable (true) o no (false). Podemos utilizar ese boolean para indicarle al sistema un día laboral o no laboral sin indicar timeSlots específicos. (Útil para integrar el módulo con Vacaciones y Permisos sin requerir utilizar Control de Asistencia).

Compatibilidad con Horarios Laborales

¿Pueden convivir ambos sistemas al mismo tiempo? - Sí. Ambos sistemas de planificación —Horarios Laborales (Work Schedule) y Gestión de Turnos (Shifts)— pueden estar activos simultáneamente. Sin embargo, existen reglas de prioridad que determinan cuál se aplica en cada caso.

A continuación, se detalla la lógica de convivencia:

1. Prioridad de Turnos

Si hay un turno asignado para un día desde el sistema de Shift Management, este tiene prioridad absoluta sobre cualquier configuración proveniente del Work Schedule.

Ejemplo: si se configuró un horario laboral desde el administrador, pero luego se asigna un turno mediante el endpoint de shifts, se considerará únicamente el turno para ese día.

2. Fallback al Horario Laboral

Si no hay turnos asignados para un día, se utilizará el horario definido en el Work Schedule (si existe).

Ejemplo: si no se envía un request de shifts para un día, el sistema tomará como referencia el horario cargado en "Horarios Laborales (Admin)". En caso de enviarse un request, siempre se priorizará esa información, tenga o no tenga timeSlots.

3. Día de Descanso (Shifts)

Si un día es marcado como descanso desde Shifts, este estado sobreescribe tanto los turnos como el horario laboral previamente asignado.

Ejemplo: esta opción permite declarar días de descanso que deben ser excluidos del cómputo de horas trabajadas, reemplazando cualquier planificación previa.

4. No se permite superposición de turnos

Si se detectan turnos superpuestos en un mismo día, el sistema devolverá un error (por ejemplo: "turnos superpuestos").

Ejemplo: enviar un turno de 09:00 a 12:00 y otro de 11:00 a 13:00 generará un error por solapamiento. En caso de recibir un segundo request sobre un mismo día, no se generará una superposición si no que una sobreescritura.

5. Sobreescritura de Turnos

Si se envían nuevos turnos para un día ya planificado, estos reemplazarán completamente los turnos anteriores para ese día.

Cada request debe contener todos los turnos válidos del día correspondiente. No se realiza un merge con turnos existentes.

6. Sugerencia de uso

Es posible utilizar los Work Schedules como base y emplear Shift Management para realizar ajustes puntuales, como modificar el horario de un día específico o marcarlo como día de descanso.

Job Positions

DELETE - /job-positions/{id} - Eliminar puesto, permite eliminar un puesto de trabajo previamente creado.

Parámetro en la URL:

id

Tipo: String

Descripción: Identificador único del puesto de trabajo a eliminar.

PATCH - /job-positions/{id} - Actualizar puesto, permite actualizar la información de un puesto de trabajo existente.

Parámetro en la URL:

id

Tipo: String

Descripción: Identificador único del puesto a actualizar.

Body:

name

Tipo: String

Descripción: Nombre del puesto.

description

Tipo: String

Descripción: Descripción general del puesto (opcional).

department

Tipo: String

Descripción: Departamento al que pertenece el puesto.

level

Tipo: String

Descripción: Nivel jerárquico del puesto (por ejemplo, "Junior", "Senior", etc.).

status

Tipo: String

Descripción: Estado del puesto. Valores posibles: **ACTIVE**, **INACTIVE**.

GET - /job-positions/bulk - Descargar plantilla para carga masiva de puestos

Permite obtener un archivo Excel con el formato requerido para la carga masiva de puestos de trabajo. No requiere parámetros ni body. El archivo que se descarga contiene las columnas necesarias para completar la información de cada puesto (nombre, descripción, departamento, nivel, etc.).

POST - /job-positions/bulk - Carga masiva de puestos

Permite crear múltiples puestos de trabajo en simultáneo mediante la carga de un archivo Excel.

Body (formato multipart/form-data):

file

Tipo: string (\$binary)

Descripción: Archivo Excel previamente completado siguiendo la plantilla descargada desde el endpoint **/job-positions/bulk**

GET - /job-positions/members/bulk - Descargar plantilla para asignación masiva de puestos a usuarios

Permite obtener un archivo Excel con el formato necesario para asignar múltiples usuarios a puestos de trabajo. No requiere parámetros ni body. El archivo que se descarga incluye las columnas necesarias para relacionar cada usuario con su puesto (por ejemplo: **employeeId**, **jobPositionId**, etc.).

POST - /job-positions/members/bulk - Asignación masiva de puestos a usuarios, permite asignar múltiples usuarios a diferentes puestos mediante la carga de un archivo Excel.

Body (formato multipart/form-data):

file

Tipo: string (\$binary)

Descripción: Archivo Excel completado según la plantilla descargada desde el endpoint **GET/job-positions/members/bulk**

Departments

GET /public/departments - Este endpoint permite obtener el listado de todos los departamentos existentes en Humand.

Body:

departments

Tipo: Array

Descripción: Lista de departamentos a crear. Cada departamento puede tener los siguientes campos:

name

Tipo: String

Descripción: Nombre del departamento. Campo obligatorio.

description

Tipo: String

Descripción: Descripción del departamento. Campo opcional.

Un ejemplo de json sería el siguiente:

```
{
  "departments": [
    {
      "name": "Ventas",
      "description": "Equipo encargado de ventas"
    },
    {
      "name": "Marketing",
      "description": "Equipo de marketing digital"
    }
  ]
}
```

GET /public/departments/{id} - Obtiene un departamento específico según su ID.

Parámetros de ruta:

id

Tipo: Number

Descripción: ID del departamento que se desea consultar.

PATCH /public/departments/{id} - Actualiza la información de un departamento específico.

Parámetros de ruta:

id

Tipo: Number

Descripción: ID del departamento que se desea modificar.

Body:

name

Tipo: String

Descripción: Nuevo nombre del departamento.

description

Tipo: String

Descripción: Nueva descripción del departamento. Campo opcional.

Un ejemplo de json sería el siguiente:

```
{
  "name": "Ventas Internacionales",
  "description": "Ventas en el exterior"
}
```

DELETE /public/departments/{id} - Elimina un departamento específico por su ID.

Parámetros de ruta:

id
Tipo: Number
Descripción: ID del departamento que se desea eliminar.

GET /public/departments/{id}/members - Obtiene los colaboradores asignados a un departamento específico.

Parámetros de ruta:

id
Tipo: Number
Descripción: ID del departamento cuyos miembros se desean consultar.

¿Cómo puedo saber qué usuarios están asignados a los departamentos?

1. Hacer una petición al endpoint **GET /public/departments**
2. Guardar los IDs
3. Hacer una petición por cada ID a **GET /public/departments/{id}/members**

POST /public/departments/{id}/members - Asigna múltiples usuarios a un departamento específico.

Parámetros de ruta:

id
Tipo: Number
Descripción: ID del departamento al que se desea agregar usuarios.

Body:

employeeInternalIds
Tipo: Array de String
Descripción: Lista de IDs internos de los empleados a asignar al departamento.

Un ejemplo de json sería el siguiente:

```
{
  "employeeInternalIds": [
    "abc123",
    "def456"
  ]
}
```

¿Cómo puedo asignar masivamente usuarios a departamentos?

1. Obtener los IDs con **GET /public/departments**
2. Hacer un POST con los **employeeInternalIds** para cada ID

DELETE /public/departments/{id}/members - Elimina múltiples usuarios de un departamento. Máximo 100 IDs por petición.

Parámetros de ruta:

id
Tipo: Number
Descripción: ID del departamento del que se desea eliminar usuarios.

Body:

employeeInternalIds
Tipo: Array de String
Descripción: Lista de IDs internos de empleados a eliminar del departamento.

Un ejemplo de json sería el siguiente:

```
{
  "employeeInternalIds": [
    "abc123",
    "def456"
  ]
}
```

¿Cómo puedo eliminar usuarios masivamente de los departamentos?

1. Obtener los IDs con **GET /public/departments**
2. Enviar un DELETE con los **employeeInternalIds** a cada departamento

PUT /public/departments/members/{employeeInternalId} - Actualiza el departamento al que pertenece un colaborador. Un colaborador solo puede estar en un departamento.

Parámetros de ruta:

employeeInternalId

Tipo: String

Descripción: ID interno del colaborador cuyo departamento se desea actualizar.

Body:

departmentId

Tipo: Number

Descripción: ID del nuevo departamento.

Ejemplo de json:

```
{
  "departmentId": 121967
}
```

¿Cómo puedo actualizar el departamento al que pertenece un usuario?

Un usuario puede estar en un solo departamento.

- Para cambiarlo, hacer un **PUT** a **/public/departments/members/{employeeInternalId}** con el nuevo **departmentId**.

Soporte y Contacto

Si tiene alguna pregunta o necesita asistencia adicional, no dude en ponerse en contacto con nuestro equipo de soporte vía mail o por el grupo de Whatsapp.

Mail: ayuda@humand.co

Conclusión

Este manual proporciona una introducción completa a la API y su uso. Esperamos que esta guía te ayude a integrar y utilizar nuestra API de manera efectiva.

¡Gracias por elegir Humand!

humand

¡Gracias por elegir
Humand!

Fecha de edición: mayo 2025